

Shortest Path Algorithm

Slide 1 — What problem does it solve?

Given a weighted graph, find the path between two nodes with minimum total cost.

- Real-world uses: GPS navigation, network routing, logistics planning
- Input: Graph $G = (V, E)$ with edge weights $w(u, v) \geq 0$
- Output: The path from source s to every vertex with minimum total weight
- Key algorithm: **Dijkstra's Algorithm** (single-source shortest path)

Graph terms

- **Vertex (node)**: a point in the graph
- **Edge**: a connection between two vertices
- **Weight**: the cost of traversing an edge

Why not BFS?

BFS finds shortest path by **hops**, not by **cost**.
Dijkstra handles weighted edges correctly.

Shortest Path Algorithm

Slide 2 — Dijkstra's Algorithm: Step-by-Step

Key idea: Greedily pick the unvisited vertex with the smallest known distance, then “relax” its neighbors.

1. **Initialize** — Set $\text{dist}[\text{source}] = 0$; all others $= \infty$. Mark all vertices unvisited.
2. **Pick** — Select unvisited vertex $\mathbf{u}$ with the smallest $\text{dist}[\mathbf{u}]$. (Use a min-heap for efficiency.)
3. **Relax neighbors** — For each neighbor $\mathbf{v}$ of $\mathbf{u}$: if $\text{dist}[\mathbf{u}] + w(\mathbf{u}, \mathbf{v}) < \text{dist}[\mathbf{v}]$, update $\text{dist}[\mathbf{v}] = \text{dist}[\mathbf{u}] + w(\mathbf{u}, \mathbf{v})$.
4. **Mark visited** — Mark $\mathbf{u}$ as visited. A visited vertex's shortest path is finalized.

5. Repeat — Go back to step 2 until all vertices are visited or the priority queue is empty.

Complexity: Using a binary min-heap $\rightarrow O((V + E) \log V)$

Using a Fibonacci heap $\rightarrow O(E + V \log V)$ — optimal for dense graphs

Shortest Path Algorithm

Slide 3 — Trace Example (source = A)

Graph edges: $A \rightarrow B(4)$, $A \rightarrow C(2)$, $C \rightarrow B(1)$, $B \rightarrow D(5)$, $C \rightarrow D(8)$, $D \rightarrow E(2)$, $B \rightarrow E(3)$

Step	Visit	dist[A]	dist[B]	dist[C]	dist[D]
Init	—	0	∞	∞	∞
1	A	0	4	2	∞
2	C	0	3 (via C)	2	10
3	B	0	3	2	8 (via B)
4	D	0	3	2	8

Shortest paths from A:

- $A \rightarrow B$: $A \rightarrow C \rightarrow B$ (cost 3)
- $A \rightarrow C$: $A \rightarrow C$ (cost 2)
- $A \rightarrow D$: $A \rightarrow C \rightarrow B \rightarrow D$ (cost 8)

Key insight: Once a vertex is marked visited, its shortest distance is final – it will never decrease further.